Digital Thermometer Using PT100

Sushmitha Ankireddy (EE26BTECH11229)

Vangara Keerthana (EE26BTECH11232)

Pareddy Anvidha Reddy (EE26BTECH11221)

1. Project Goal

Our aim is to build a digital thermometer using a PT100 temperature sensor and an Arduino Uno. The temperature should be shown on a 16x2 LCD. We have an Arduino Uno, a PT100 sensor, resistors, an LCD, a potentiometer, a perfboard and jumper wires.

Components used:

S.No.	**Component**	Quantity
1.	Arduino Uno	1
2.	PT100 temperature sensor	1
3.	180 Ω resistor	1
4.	16x2 LCD display	1
5.	Potentiometer	1
6.	Perfboard	1
7.	Jumper wires	As required
8.	Header strips	As required
9.	Soldering iron and solder	1 set
10.	USB cable	1

Purpose of Each Component

- **Arduino Uno:** The main controller of the project. It reads the sensor voltage, runs our code to calculate the temperature, and sends the result to the LCD. It also supplies power to the circuit.
- **PT100 temperature sensor:** The sensor that detects the temperature. Its resistance changes with temperature, which is what we measure.
- **180 Ω resistor:** The reference resistor connected in series with the PT100. Together they convert the sensor's changing resistance into a voltage that the Arduino can read.
- **16x2 LCD display:** Displays the measured temperature on two rows of 16 characters each.
- **Potentiometer:** Adjusts the contrast of the LCD so the text is clearly visible. It is not involved in measuring the temperature.
- **Perfboard:** The board on which the circuit is built and soldered. It holds all the components in place and makes the connections permanent.
- **Jumper wires:** Used to make the electrical connections between the components.
- **Header strips:** Soldered onto the perfboard so the Arduino and wires plug in firmly and stay removable.

Circuit Connections:

S.No.	Component	Pin / Terminal	Connected To
1	Arduino Uno	5V	5 V rail

2	Arduino Uno	GND	GND rail
3	180 Ω resistor	Terminal 1	5 V rail
4	180 Ω resistor	Terminal 2	A0 junction (common point with the PT100)
5	PT100	Wire 1	A0 junction
6	PT100	Wire 2	GND rail
7	Arduino Uno	A0	A0 junction
8	Potentiometer	Outer pin 1	5 V rail
9	Potentiometer	Middle pin	LCD V0 (pin 3)
10	Potentiometer	Outer pin 2	GND rail

11	LCD	VSS (pin 1)	GND rail
12	LCD	VDD (pin 2)	5 V rail
13	LCD	V0 (pin 3)	Potentiometer middle pin
14	LCD	RS (pin 4)	Arduino D12
15	LCD	RW (pin 5)	GND rail
16	LCD	E (pin 6)	Arduino D11
17	LCD	D0 to D3 (pins 7 to 10)	Not connected
18	LCD	D4 (pin 11)	Arduino D5
19	LCD	D5 (pin 12)	Arduino D4
20	LCD	D6 (pin 13)	Arduino D3
21	LCD	D7 (pin 14)	Arduino D2

22	LCD	A, backlight anode (pin 15)	5 V rail
23	LCD	K, backlight cathode (pin 16)	GND rail

2. Initial Research and Understanding

We started researching the project, focusing on the theoretical principles and how the sensor works. A PT100 is a temperature sensor whose resistance changes with temperature. It has a resistance of 100 Ω at 0 °C and increases by about 0.385 Ω for every 1 °C rise, which gives the temperature formula

(Resistance - 100) / 0.385 = Temperature

The Arduino cannot measure resistance directly, so this change in resistance has to be converted into a voltage first.

3. Choosing the Circuit: Wheatstone Bridge or Potentiometer?

While researching, we found that some sources use a Wheatstone bridge, and we were also unsure about the role of the potentiometer. After going through both, we understood the following:

- The **potentiometer** is not used for measuring temperature. In our circuit, it only adjusts the contrast of the LCD.
- The **Wheatstone bridge** gives more precise readings, but it needs extra components like an external ADC (ADS1115), which we do not have.

Based on this, we decided to use a **voltage divider**. In this circuit, the PT100 is connected in series with a fixed resistor, and the voltage at the middle point is read by the Arduino's analog pin A0. This suits our available components and is simpler to build.

4. How Our Circuit Works

The Arduino's ADC converts the voltage at A0 into a number between 0 and 1023. Our code then does the following:

1. Converts the ADC number into a voltage using $V_{out} = \text{ADC} \times \left(\frac{5.0}{1023}\right)$.
2. Uses the voltage divider formula to calculate the PT100's resistance: $R_{PT100} = R_{REF} \times \left(\frac{V_{out}}{5.0 - V_{out}}\right)$.
3. Converts the resistance into temperature using the linear relation formula: $\text{Temperature} = \frac{R_{PT100} - 100}{0.385}$.
4. Displays the temperature on the LCD.

5. Building the Circuit and Initial Challenges

We started by setting up a **trial circuit on the breadboard**. Our project is supposed to be built as a soldered circuit, but we wanted to test the connections and the code first, so we decided to solder it later, once the circuit was working properly.

In this circuit, the PT100 and a fixed resistor form the voltage divider, and the junction between them is connected to pin A0 of the Arduino. The LCD is connected in 4-bit mode, and the potentiometer is wired to the LCD's contrast pin. Before wiring, we prepared a complete table of all the connections, and we followed it step by step so that nothing was missed.

While building this trial circuit, we faced a few challenges.

Resistor value: We first planned to use a 100 Ω reference resistor, but the one we had was 180 Ω. We updated the value in our code (`R_REF = 180.0`) so that the calculations match the actual resistor.

Wrong temperature reading: After uploading the code, the LCD showed about **-60 °C to -67 °C** at room temperature, when it should have shown around 25 to 30 °C. This was our main challenge at this stage, and most of our later work was about finding and fixing it.

6. Debugging the Wrong Reading

After uploading the code, the LCD showed a temperature of about -60 °C to -67 °C at room temperature, which is clearly wrong. We wanted to find out whether the problem was in our code or in the circuit, so we decided to look at the raw values instead of only the final temperature.

For this, we wrote a separate debug program. Instead of showing the temperature, it shows the **raw ADC value** and the **calculated resistance of the PT100** on the LCD. The ADC value is the number the Arduino gets after converting the voltage at pin A0, so it tells us what the Arduino is actually reading from the circuit.

With this program, we got an ADC value of about **259** and a resistance of about **61 Ω**.At room temperature, we expected an ADC value of about **386** and a resistance of about **109 Ω**. . Both values were much lower than they should be.

This helped us understand something important. The numbers on the LCD matched each other, which means the calculations in our code were working properly. The wrong temperature was not because of a mistake in the formula. It meant the voltage actually reaching the Arduino was lower than it should be, so the problem had to be somewhere in the circuit.

We also checked the values of the PT100 and the resistor, and both were fine. So the sensor and the resistor were not the cause, and we had to look at the rest of the circuit.

7. Narrowing Down the Problem

Once we knew the problem was in the circuit and not in the code, we started to think about where the voltage could be getting lost. A voltage divider depends completely on the connections around it, and even a small problem at the junction connected to A0 can change the voltage the Arduino reads.

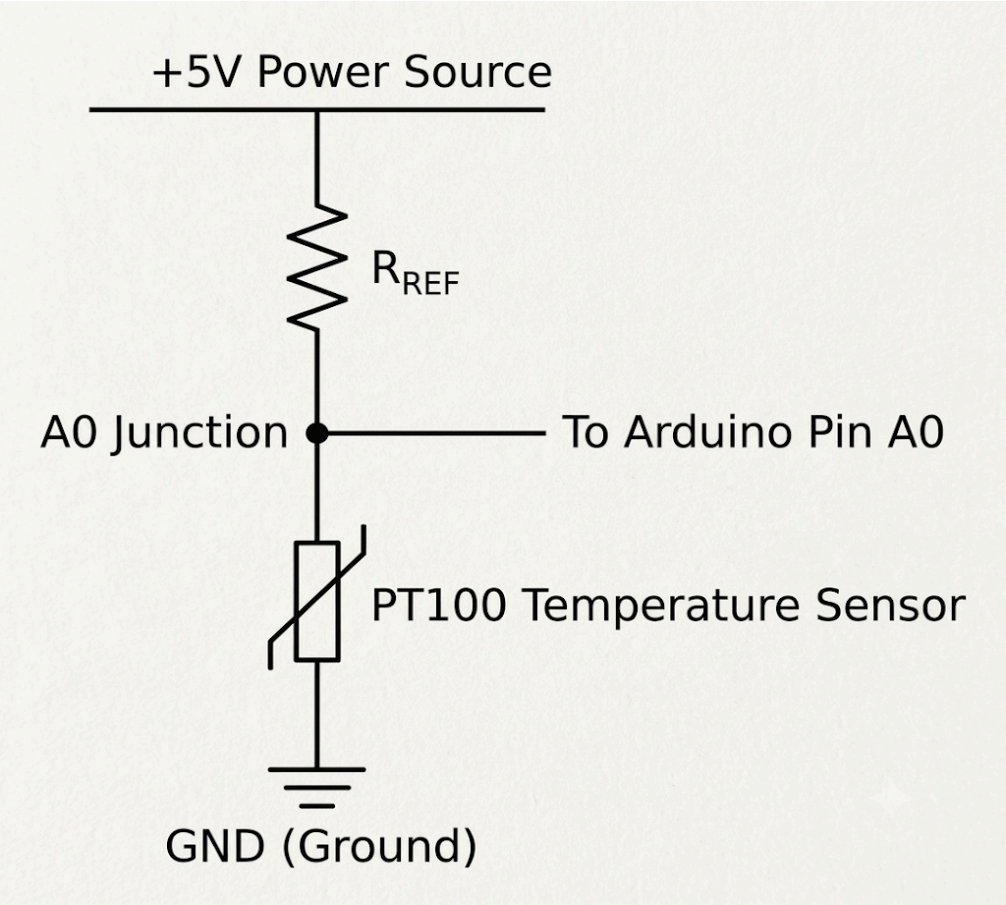
Our next step was to go through the circuit carefully. We planned to recheck every connection against our connection table and to measure the voltage at the A0 junction directly, so that we could compare it with the value the Arduino was reading. We also planned to look closely at the breadboard and the jumper wires, since loose or faulty contacts are a common cause of wrong readings in circuits like this.

8. Explanation of Voltage at the A0 Junction

The voltage divider circuit controls how voltage is split between the reference resistor and the PT100 sensor before being measured by the Arduino:

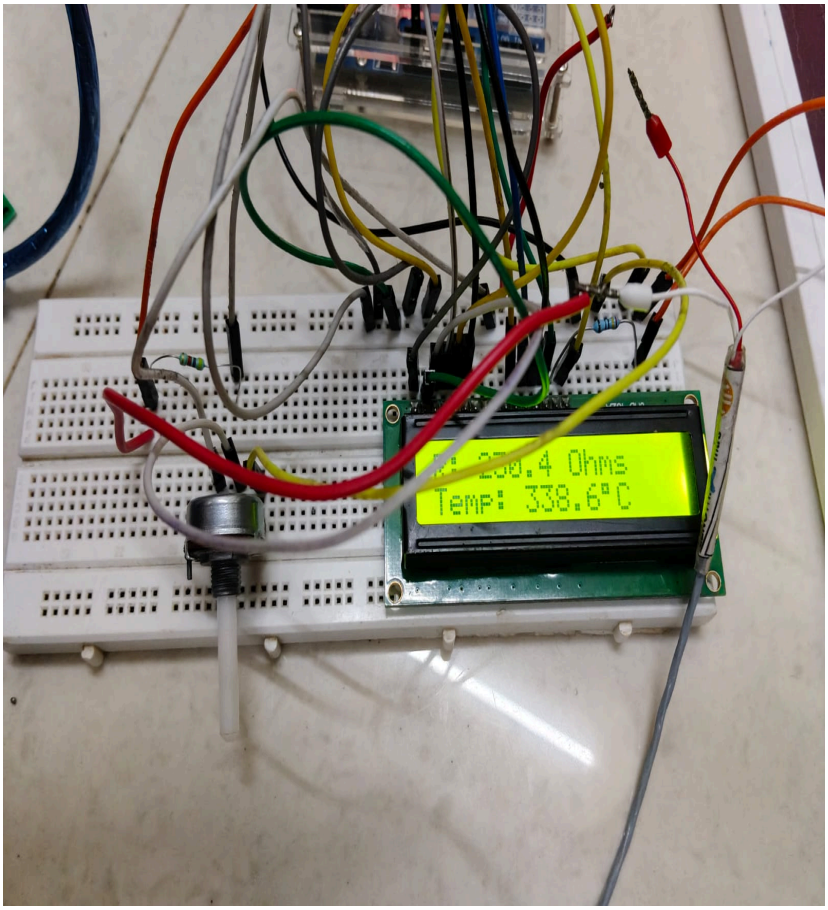
- **A0 Junction:** The "A0 junction" is simply the point in the circuit where the reference resistor (R_REF) connects to the PT100 sensor, and a wire leads directly to analog pin A0 on the Arduino.
- **Current Flow:** A 5V power supply from the Arduino flows first through the 120 Ω reference resistor (R_REF), passes through the PT100 sensor, and then completes the circuit to Ground (GND).
- **Voltage Reading at A0:** Pin A0 measures the voltage level specifically at the middle junction between R_REF and the PT100. As the temperature fluctuates, the resistance of the PT100 changes.

This shifts the ratio of voltage dropped across R_REF versus the PT100, causing a proportional change in the voltage measured at pin A0.



9. Testing a Different Breadboard

After getting the wrong reading of around 340 °C , we first wanted to check if the breadboard itself was the problem. Breadboards can have loose contacts or broken rows, which can change the voltage without us noticing. So we moved the whole circuit onto a different breadboard and wired everything again from the start, following our connection table.



Even after this, the LCD showed about **-90 °C** at room temperature. The reading was still wrong, and it was in the same direction as before. This told us the breadboard was not the cause, so we could rule it out and look at other parts of the circuit.

10. Choosing the Reference Resistor

Next, we did more research to understand the role of the reference resistor in the voltage divider. We were using a **180 Ω** resistor, which is quite low. A low resistance lets more current flow through the circuit, and this current also passes through the PT100. When current flows through a PT100, it produces a small amount of heat in the sensor itself. This is called **self-heating**, and it can slightly change the readings.

Because of this, we wanted to try a higher resistance, so we picked **2.2 kΩ** and **4.7 kΩ**. The logic is simple: a higher resistor means less current flows through the circuit, so the PT100 heats up less.

But we also saw that there is a trade-off. The voltage at A0 is given by the voltage divider formula:

V = 5 × R_PT100 / (R_ref + R_PT100)

When the reference resistor is larger, the voltage at A0 becomes smaller, and it changes by a much smaller amount for each °C. The Arduino's ADC can only detect voltage changes in steps of about 4.9 mV. So, with a smaller change in voltage per °C, one ADC step covers more degrees. This means the reading becomes coarser. The size of one ADC step in each case is:

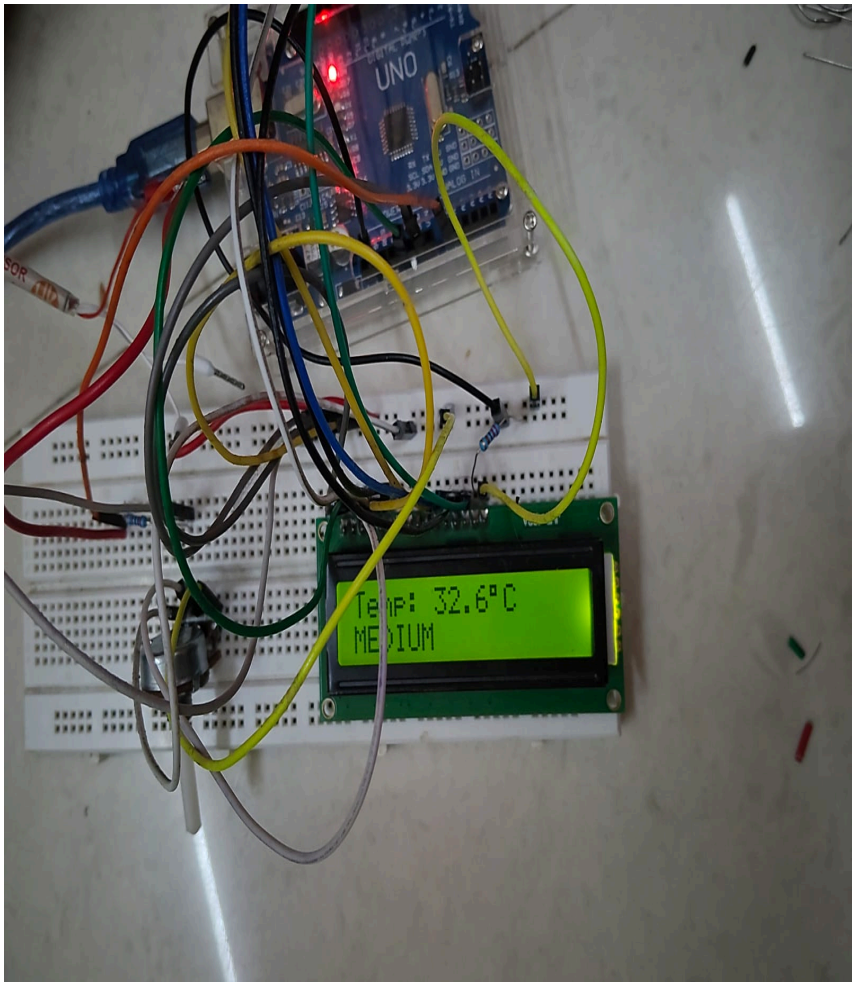
- **180 Ω:** about **1.2 °C** per step
- **2.2 kΩ:** about **6 °C** per step
- **4.7 kΩ:** about **12.5 °C** per step

So a higher resistor reduces the heating, but it also makes the reading less precise.

To test this, we changed the wiring using new jumper wires and tried all three resistors one after another, updating the reference resistor value in the code each time. We got a sensible temperature reading in all three cases. However, with 2.2 kΩ and 4.7 kΩ the reading was **fluctuating a lot**. This matches the calculation, because even a tiny noise in the voltage can move the ADC by one step, and with a large resistor one step means a jump of several degrees. With 120 Ω, one step is only about 1 °C, so the reading stays stable. Because of this, we decided to continue with the **180 Ω** resistor.

11. First Sensible Readings

After changing the wiring with new jumper wires, the LCD finally started showing a temperature close to the actual room temperature, instead of the



large negative values we saw earlier. This happened with all three resistor values we tested, which suggests that the earlier wrong readings of 340 °C to -90 °C were mainly caused by poor contact in the old jumper wires and not by the resistor value. The only difference between the resistors was that the readings with 2.2 kΩ and 4.7 kΩ fluctuated a lot, which is why we finally went ahead with 180 Ω.

12. Moving to Soldering

Our project was supposed to be built as a soldered circuit. The breadboard was only used in the beginning to test the connections and the code. After changing the wiring with new jumper wires and finalising the **180 Ω** reference resistor, the LCD showed a stable temperature close to the

actual room temperature. This told us the circuit design and the code were working properly, so we moved on to soldering the circuit onto a perfboard.

We started by soldering the main parts of the circuit onto the perfboard. This included the reference resistor, the PT100 connections and the wire going to the A0 pin.

Since our circuit has an Arduino, we could not solder anything directly onto the Arduino board. So we used **header strips**. A header strip is a row of metal pins or sockets that is soldered onto the perfboard. The Arduino and the wires then plug into these headers. This way the connections are firm, but the Arduino stays removable and can be taken out whenever needed.

We then continued the soldering with the header strips in place. The reference resistor, the PT100 connections and the wires to the LCD and the potentiometer were soldered onto the perfboard, and the Arduino was connected through the headers without any soldering on the Arduino itself.

The pins on the header strips are very close to each other, so a few of them got shorted together by the solder when we started, but we were able to separate them easily.

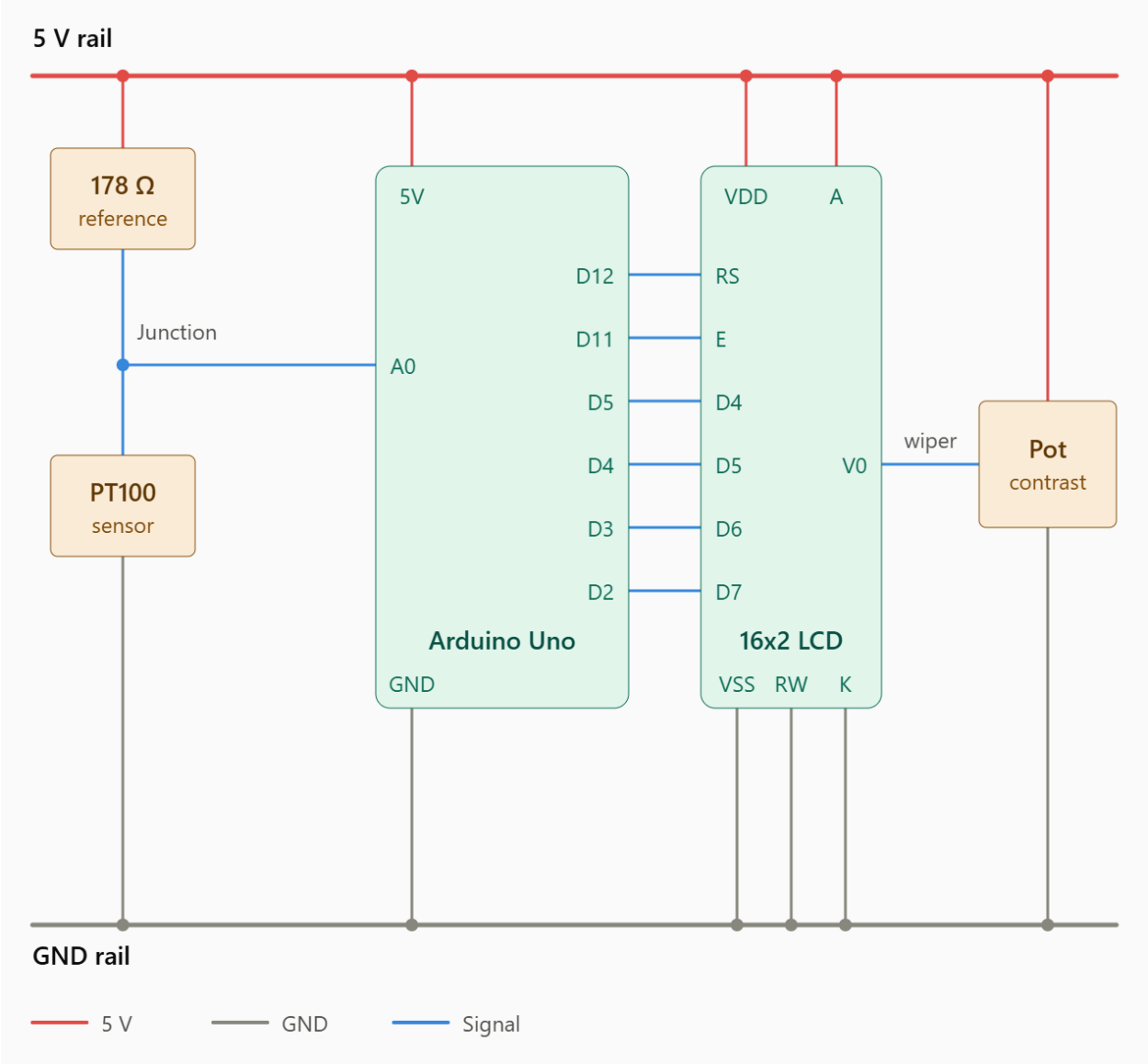
Many of the connections in our circuit have to go to either the 5 V supply or ground (GND). The reference resistor, one end of the potentiometer and the LCD's power and backlight pins all need 5 V. In the same way, the PT100, the other end of the potentiometer and several LCD pins (including the backlight cathode) need to be connected to GND. Running a separate wire from the Arduino to each of these points would have made the board crowded and difficult to solder.

So we created a **5 V rail** and a **GND rail** on the perfboard. Each rail is a common line connected to the matching pin on the Arduino, and every component that needs 5 V or GND is connected to its rail. This kept the wiring neat, reduced the number of wires coming from the Arduino, and made it easier to check the connections later.

13. Soldered Circuit Working

After finishing the soldering, we powered the circuit again and checked the LCD. The thermometer gave the **correct temperature readings** even after the circuit was moved from the breadboard to the perfboard. This confirmed that the soldered connections, the header strips and the 5 V and GND rails were all working as planned, and that the code and the reference resistor value were set correctly.

Circuit Diagram



14. Final Code

```
//
============================================================
============

// PT100 Digital Thermometer - Arduino Uno + 16x2 LCD

// 17-point linear least-squares calibration (Updated Readings)

//

// T_cal = CAL_A * T_raw + CAL_B

//
============================================================
============


#include <LiquidCrystal.h>

#include <math.h>


LiquidCrystal lcd(12, 11, 5, 4, 3, 2);


// ---------- Sensor settings ----------

const float R_REF = 178.0;


// ---------- Calibration data ----------

// X_OURS = raw PT100 temperature in Celsius

// Y_ACTUAL = reference thermometer temperature in Celsius
```

```
const int N = 17;

const float X_OURS[N] = {
  56.05, 60.63, 61.92, 64.55, 67.18,
  69.69, 73.90, 76.57, 73.90, 71.18,
  76.84, 77.93, 78.02, 78.27, 79.31,
  79.31, 79.31
};

const float Y_ACTUAL[N] = {
  55.3, 63.9, 64.3, 71.9, 72.9,
  74.8, 82.9, 84.3, 82.9, 80.8,
  85.5, 91.2, 92.8, 93.2, 94.2,
  95.1, 96.8
};

float CAL_A = 1.0;
float CAL_B = 0.0;

// ---------- PT100 constants (IEC 60751) ----------
const float R0 = 100.0;
```

```
const float A  = 3.9083e-3;
const float B  = -5.775e-7;

// ---------- Settings ----------
const int SAMPLES = 256;
const float SMOOTH = 0.85;

float tempSmooth = NAN;

// ---------- Calculate least-squares coefficients ----------
void leastSquares()
{
  float sx = 0.0;
  float sy = 0.0;

  for (int i = 0; i < N; i++)
  {
    sx += X_OURS[i];
    sy += Y_ACTUAL[i];
  }
```

```
float xm = sx / N;
float ym = sy / N;

float sxy = 0.0;
float sxx = 0.0;

for (int i = 0; i < N; i++)
{
  sxy += (X_OURS[i] - xm) * (Y_ACTUAL[i] - ym);
  sxx += (X_OURS[i] - xm) * (X_OURS[i] - xm);
}

if (sxx > 0.0)
{
  CAL_A = sxy / sxx;
  CAL_B = ym - CAL_A * xm;
}

Serial.println();
Serial.println("===== CALIBRATION RESULTS =====");
```

```
  Serial.print("Calibration points: ");
  Serial.println(N);

  Serial.print("CAL_A = ");
  Serial.println(CAL_A, 6);

  Serial.print("CAL_B = ");
  Serial.println(CAL_B, 6);

  Serial.print("Equation: T_cal = ");
  Serial.print(CAL_A, 6);
  Serial.print(" * T_raw + ");
  Serial.println(CAL_B, 6);

  Serial.println("================================");
}

// ---------- Read averaged ADC ----------
float readADC()
{
  long sum = 0;
```

```
  for (int i = 0; i < SAMPLES; i++)
  {
    sum += analogRead(A0);
    delayMicroseconds(300);
  }

  return sum / (float)SAMPLES;
}

// ---------- Show errors ----------
void showError(const char *line1, const char *label, float value)
{
  Serial.print("ERROR: ");
  Serial.print(line1);
  Serial.print(" ");
  Serial.print(label);
  Serial.println(value, 1);

  lcd.setCursor(0, 0);
  lcd.print(line1);
```

```
  lcd.setCursor(0, 1);
  lcd.print(label);
  lcd.print(value, 1);
  lcd.print("          ");

  tempSmooth = NAN;
  delay(500);
}

// ---------- Setup ----------
void setup()
{
  Serial.begin(9600);
  lcd.begin(16, 2);

  lcd.clear();
  lcd.print("PT100 Digital");
  lcd.setCursor(0, 1);
  lcd.print("Calibrating...");
  delay(1500);
```

```
  leastSquares();

  lcd.clear();
  lcd.print("Calibration OK");
  delay(1000);
  lcd.clear();

  Serial.println();
  Serial.println("Starting PT100...");
  Serial.println("ADC | Resistance | Raw Temp | Cal Temp | Display");
}

// ---------- Main loop ----------
void loop()
{
  float adc = readADC();

  if (adc < 10.0)
  {
    showError("PT100 shorted?  ", "ADC: ", adc);
```

```
  return;
}

if (adc > 1010.0)
{
  showError("PT100 open?     ", "ADC: ", adc);
  return;
}

// Calculate resistance
float rpt = R_REF * adc / (1023.0 - adc);

if (rpt < 60.0 || rpt > 400.0)
{
  showError("R out of range  ", "R: ", rpt);
  return;
}

// Convert resistance to raw temperature
float x = rpt / R0 - 1.0;
float disc = A * A + 4.0 * B * x;
```

```
if (disc < 0.0)
{
  showError("Calc error      ", "R: ", rpt);
  return;
}

float tempRaw = (2.0 * x) / (A + sqrt(disc));

// Apply least-squares calibration
float tempC = CAL_A * tempRaw + CAL_B;

// Smooth output
if (isnan(tempSmooth))
{
  tempSmooth = tempC;
}
else
{
  tempSmooth =
      SMOOTH * tempSmooth + (1.0 - SMOOTH) * tempC;
```

```
}

// Serial Monitor output
Serial.print("ADC=");
Serial.print(adc, 1);

Serial.print("  R=");
Serial.print(rpt, 2);
Serial.print(" ohm");

Serial.print("  T_raw=");
Serial.print(tempRaw, 2);

Serial.print("  T_cal=");
Serial.print(tempC, 2);

Serial.print("  T=");
Serial.println(tempSmooth, 2);

// LCD: temperature
lcd.setCursor(0, 0);
```

```
  lcd.print("Temp: ");
  lcd.print(tempSmooth, 1);
  lcd.print((char)223);
  lcd.print("C     ");

  // LCD: resistance
  lcd.setCursor(0, 1);
  lcd.print("R: ");
  lcd.print(rpt, 1);
  lcd.print(" ohm     ");

  delay(300);
}
```

LSQ code:

```
import numpy as np

# raw pt100 readings
x_ours = [
    56.05, 60.63, 61.92, 64.55, 67.18,
    69.69, 73.90, 76.57, 73.90, 71.18,
    76.84, 77.93, 78.02, 78.27, 79.31,
```

```
    79.31, 79.31
]

# actual reference thermometer temperatures
y_actual = [
    55.3, 63.9, 64.3, 71.9, 72.9,
    74.8, 82.9, 84.3, 82.9, 80.8,
    85.5, 91.2, 92.8, 93.2, 94.2,
    95.1, 96.8
]

x = np.array(x_ours)
y = np.array(y_actual)

# 1st degree polynomial linear least squares
slope, intercept = np.polyfit(x, y, 1)

print("===== CALIBRATION RESULT =====")
print(f"CAL_A (Slope)     = {slope:.6f}")
print(f"CAL_B (Intercept) = {intercept:.6f}")
print(f"Equation: T_cal = {slope:.6f} * T_raw + {intercept:.6f}")
```

straight line equation

$$y = a \cdot x + b$$

x = raw pt100 temperature reading$y$ = calibrated temperature

a = slope constant (CAL_A)

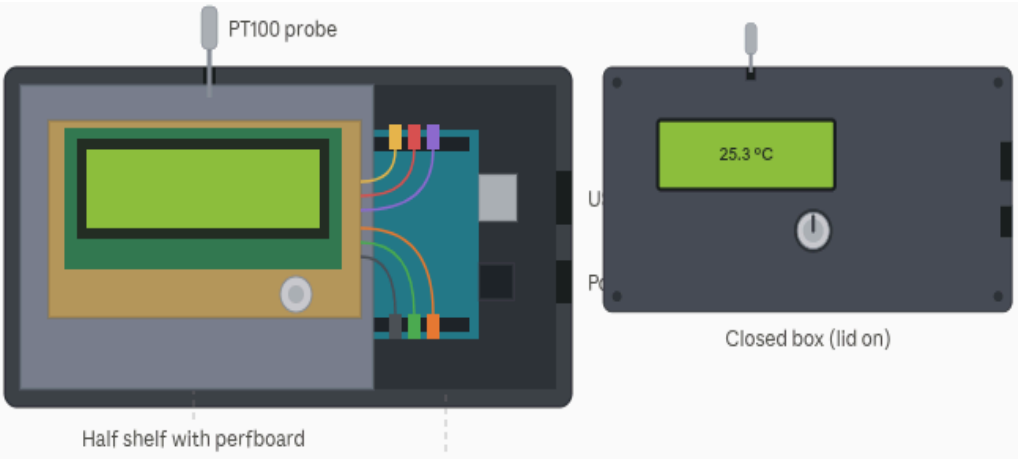
b = intercept constant (CAL_B)

15. 3D Model

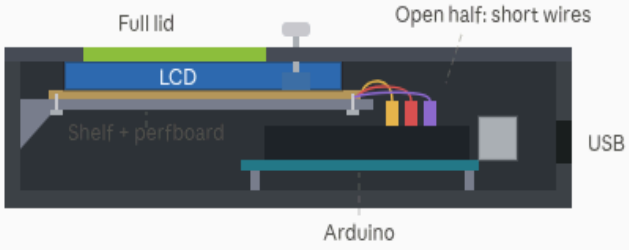
Key Design Features:

- Two-Tier Layout: The Arduino Uno (in its acrylic case) sits at the bottom, while a built-in $50mm$ shelf holds our custom perfboard shield right above it.
- LCD & Potentiometer Mounts: The lid features a cut-out window with a rim that fits flush around the LCD screen, plus a recessed well so the contrast adjustment knob sits neatly inside.
- Clean Port Openings: Cutouts on the sides allow easy access for the Arduino USB cable and a dedicated $7mm$ hole for the PT100 probe wire.

Box Dimensions: $89mm \times 75mm \times 80mm$ **(**$L \times W \times H$**) with** $2.5mm$ **thick walls.**



Top view with the lid taken off



Side cutaway

Full lid on top; only the shelf is half size